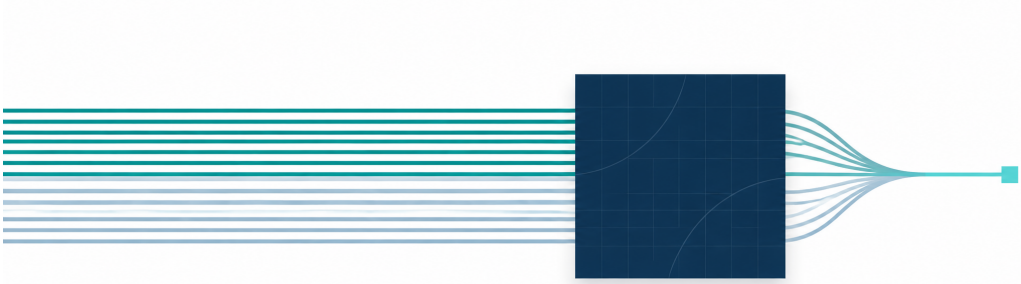
香山昆明湖 V2

自定义向量点积指令

第一阶段：环境部署与向量加法验证

设计、验证与性能评估



三阶段工程技术报告

2026 年 11 月 6 日修订

目录

第一阶段：环境部署与向量执行过程 **1**

1.1 协同验证框架与实验环境 **1**

1.2 部署与构建过程 **3**

1.2.1 资源配置与工具位置 3

1.2.2 固定源码与工具安装 4

1.2.3 集成构建与操作记录 5

1.3 共同启动输入与差分故障定位 **7**

1.4 hello 输出与三类 vadd 结果 **8**

1.4.1 输入配置与结果解释 8

1.4.2 10 月 9 日环境验证 9

1.4.3 10 月 25 日双模型复核 9

1.5 向量执行与波形观察 **10**

1.5.1 10 月 9 日事件与原始截图 10

1.5.2 10 月 25 日含时钟的调试记录 12

1.6 阶段结论与复现索引 **13**

1.6.1 源码与工具版本 13

1.6.2 T04 产物身份 14

1.6.3 文件与记录索引 14

1.7 操作日志摘录 **15**

1.7.1 10 月 9 日工作负载构建 15

1.7.2 10 月 9 日正常运行摘要 15

1.7.3 10 月 25 日生产模型输出 16

1.7.4 10 月 25 日 FST 模型输出 16

第一阶段：环境部署与向量执行过程

本阶段对应赛题的环境部署验证、操作日志和向量加法执行过程。2026 年 10 月 9 日的记录展示环境建立、启动差分修复及三类 vadd 用例；10 月 25 日的 T04 复核进一步完成英文 hello 输出、两种 EMU 模型的差分运行和含时钟的波形调试。下文按两组实验各自的源码、输入和产物说明结果，自定义 vdot 点积的设计与测试见第二阶段。

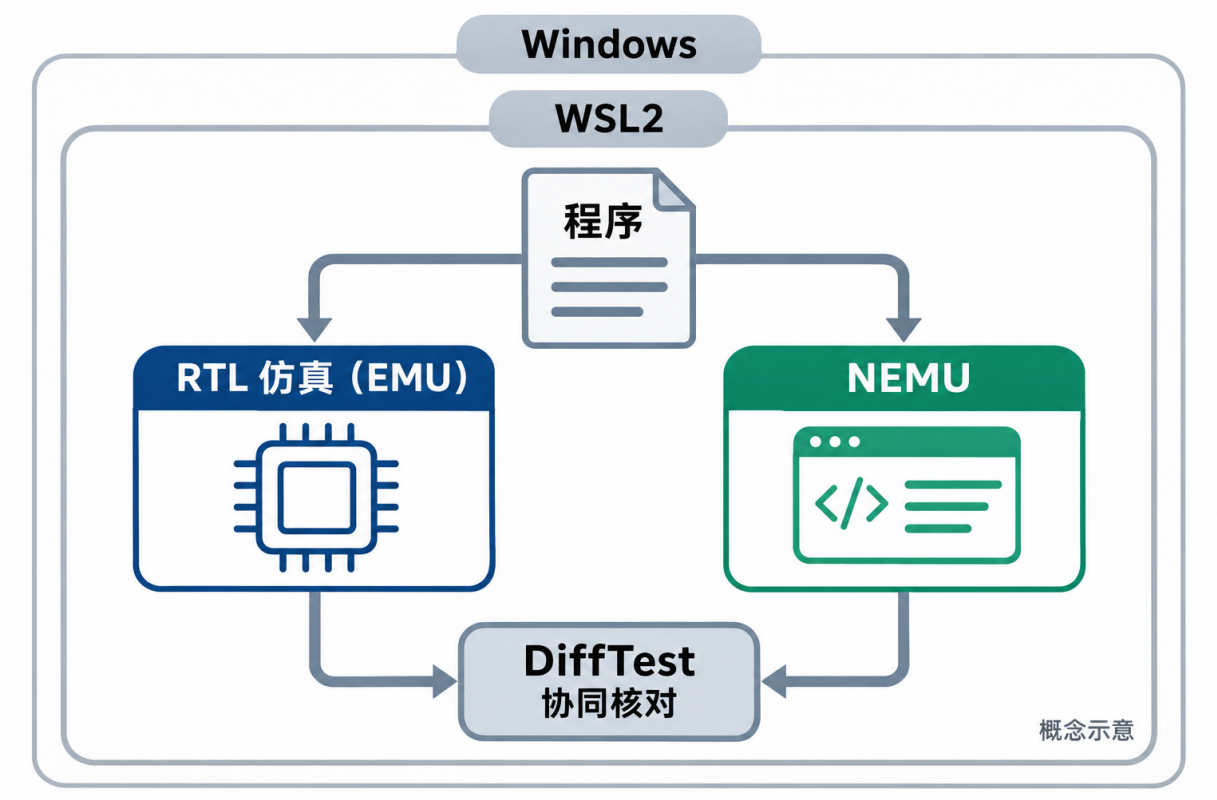
1.1 协同验证框架与实验环境

香山 EMU、NEMU 参考模型和裸机程序共同构成验证环境。EMU 模拟硬件执行，NEMU 按指令语义更新参考状态，DiffTest 在架构状态比较点核对两者。裸机程序提供可重复的输入和结果检查；共同启动 flash 把复位过程连接到程序入口。hello 检查程序启动与串口输出，vadd 则覆盖向量配置、加载、按元素加法、存储和提交。

硬件侧由 Java 与 Mill 解析 Scala/Chisel 工程，firtool 处理硬件中间表示，Verilator 与宿主 C++ 编译器生成 EMU。目标程序使用 RISC-V GCC 和 binutils 编译、链接并生成 ELF、BIN 及反汇编。NEMU 在宿主侧构建为参考库，由运行参数显式加载。各条构建链最终汇合到同一次差分运行，构建记录与运行记录通过产物摘要相连。

表 1-1 构建产物与复核入口

产物	作用	主要记录
香山 EMU	执行 DUT 仿真	硬件源码、DUT 配置、生成命令、编译工具与文件 SHA-256
NEMU 参考库	提供架构状态参考	固定源码、构建配置、实际加载路径与文件 SHA-256
程序 ELF/BIN	执行 hello 和 vadd 用例	ISA/ABI、入口、装载段、反汇编、镜像摘要
共同 flash	建立一致的启动输入	启动汇编、12 字节镜像、反汇编与显式加载参数
运行记录	关联输入、结果及结束状态	原始日志、退出码、波形、事件 CSV 与信号索引



概念示意：共同输入与软硬件状态核对。技术依据见图 1-1。

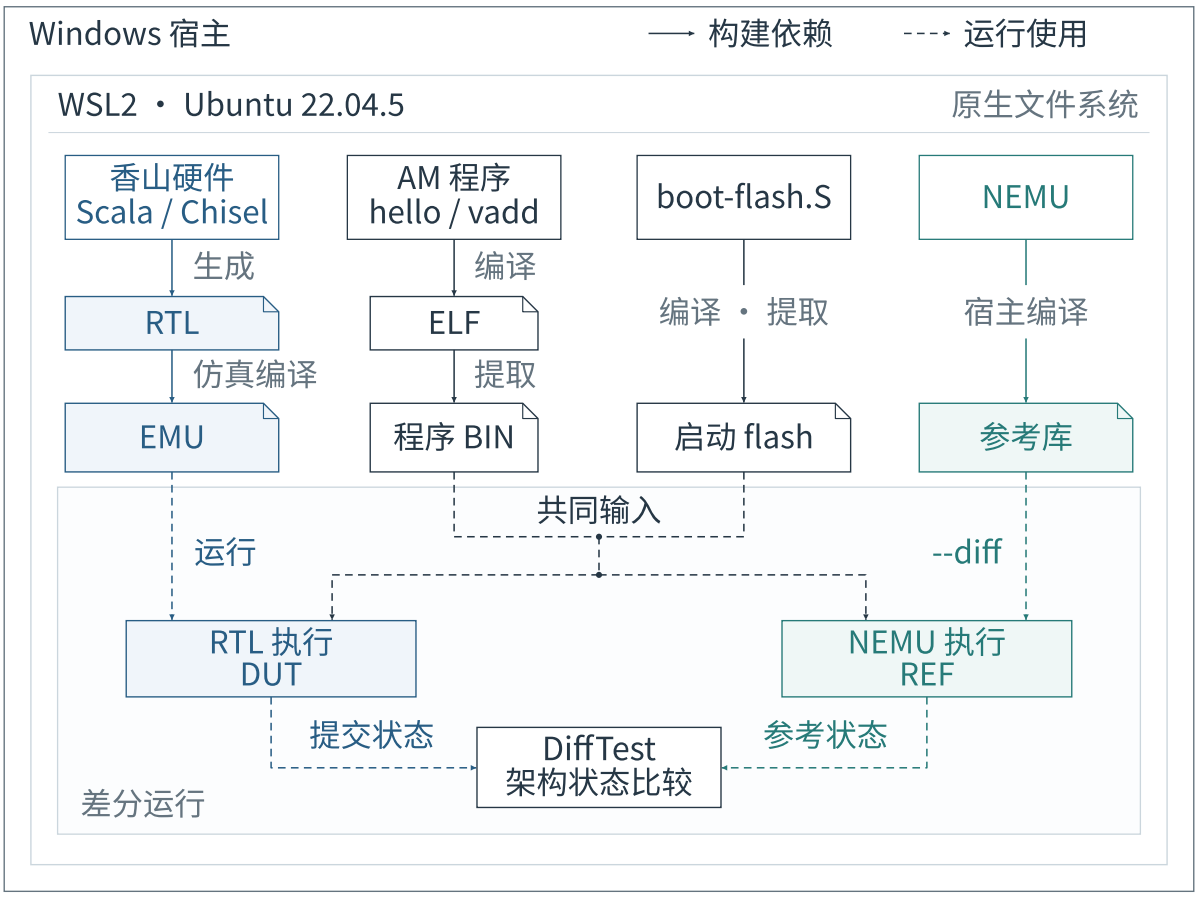


图 1-1 开发环境与软硬件协同验证框架

Windows 宿主中的 WSL2 承载构建与差分运行。实线表示构建依赖，虚线表示运行使用关系；程序 BIN 与启动 flash 作为共同输入，EMU 与 NEMU 分别提供 DUT 和参考状态。

10 月 9 日实验在 Windows 下的 WSL2 Ubuntu 22.04.5 运行，宿主处理器为 Ryzen 7 7840H，可用物理内存约 27.7GiB。WSL 配置 20GB 内存和 16GB swap，构建与仿真放在 WSL 原生文件系统。DUT 为 MinimalConfig、单核、VLEN128，原波形构建采用 EMU_THREADS=1 和 EMU_TRACE=fst。运行时显式指定 128MB RAM、共同 flash 和固定参考库。

本阶段的源码锁、完整工具版本和 T04 产物摘要集中列于本章末尾。10 月 9 日的部署基线与 10 月 25 日复用的 T17 生成物分别标识，便于准确恢复对应实验。

1.2 部署与构建过程

1.2.1 资源配置与工具位置

WSL 资源设置位于 Windows 用户目录的%UserProfile%\.wslconfig：

```
[wsl2]
memory=20GB
swap=16GB
```

保存正在进行的 WSL 工作后，在 PowerShell 执行`wsl --shutdown`，重新进入 Ubuntu 并用`free -h`核对。原机器的交换文件位于 D:\WSL\wsl-swap.vhdx，其他机器按本机磁盘布局配置。内

存规划同时覆盖 JVM、硬件生成工具、C++ 编译进程和文件缓存；频繁换页会增加构建时间，因此应根据可用物理内存调整 JVM 堆与编译并发。

Windows 工作树的 Git 检查、提交与推送使用 Windows 原生 Git；Linux/WSL 原生目录使用 Linux Git。Scala/Chisel、RISC-V 程序、EMU/NEMU 和波形处理在 Linux 原生目录执行。源码与产物跨环境交换时，在接收端核对完整提交号、未提交差异、外部依赖及关键文件 SHA-256，运行输出使用独立目录。

这一安排也对应实际工程经验：2026 年 10 月 24 日 R4 发布检查中，一次从 WSL 访问 Windows 工作树的检查约 300 秒后超时；核实未提交且没有残留进程或索引锁后，Windows 原生 Git 完成了后续检查与提交。具体过程见协调记录，该记录用于说明执行位置的选择。

1.2.2 固定源码与工具安装

在团队仓库执行`bash team/scripts/prepare-dependencies.sh`，按版本锁初始化 xs-env、NEMU、nexus-am、DRAMsim3 及递归依赖。团队仓库的香山子模块使用`git clone --recurse-submodules`恢复；NEMU 的 SimPoint 按子模块记录初始化，其余下载依赖按锁定脚本检出。已有目录的版本或已跟踪修改不一致时，先保留现场并核对输入。构建前记录实际仓库根、分支、完整 HEAD、Git common-dir、修改状态与正在使用的输入输出，避免与其他任务共用可写 Git 索引或覆盖冻结产物。

基础工具的安装入口如下。这里的命令用于恢复固定环境，安装后再核对实际工具版本。

```
source team/scripts/activate.sh
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target base
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target llvm
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target mill
sudo apt-get install -y cmake ninja-build gtkwave
```

该入口来自固定 xs-env 的setup-tools.sh。软件源中的补丁版本会变化，环境检查负责记录安装后实际使用的版本。Mill 读取仓库的.mill-version 选择项目版本，Scala、Chisel 和 firtool 按项目解析结果核对。

原 JDK 安装入口使用 Oracle 21/latest 地址。团队的固定安装脚本改用 21.0.12 Linux x64 归档及官方 SHA-256，安装到用户目录：

```
export JAVA_HOME="$HOME/.local/share/graalvm-jdk-21.0.12"
bash team/scripts/install-graalvm.sh
source team/scripts/activate.sh
```

新终端沿用相同 JAVA_HOME。原/opt/graalvm-jdk-21 安装可在版本核对后继续使用；脚本保留版本不同的已有安装。参考资料为Oracle 安装说明、固定归档和官方校验值。

Verilator 从固定提交构建，显式选择已安装的 Clang，并按资源情况控制编译并发：

```
sudo apt-get install -y help2man perl autoconf flex bison
libgoogle-perftools-dev \
  libjemalloc-dev numactl perl-doc zlib1g-dev liblz4-dev libfl2 libfl-dev
mkdir -p "$HOME/riscv-tools"
git clone https://github.com/verilator/verilator.git
"$HOME/riscv-tools/verilator"
cd "$HOME/riscv-tools/verilator"
git checkout ea338be98e1e838d3518809ce8899f85a009963c
unset VERILATOR_ROOT
autoconf
```

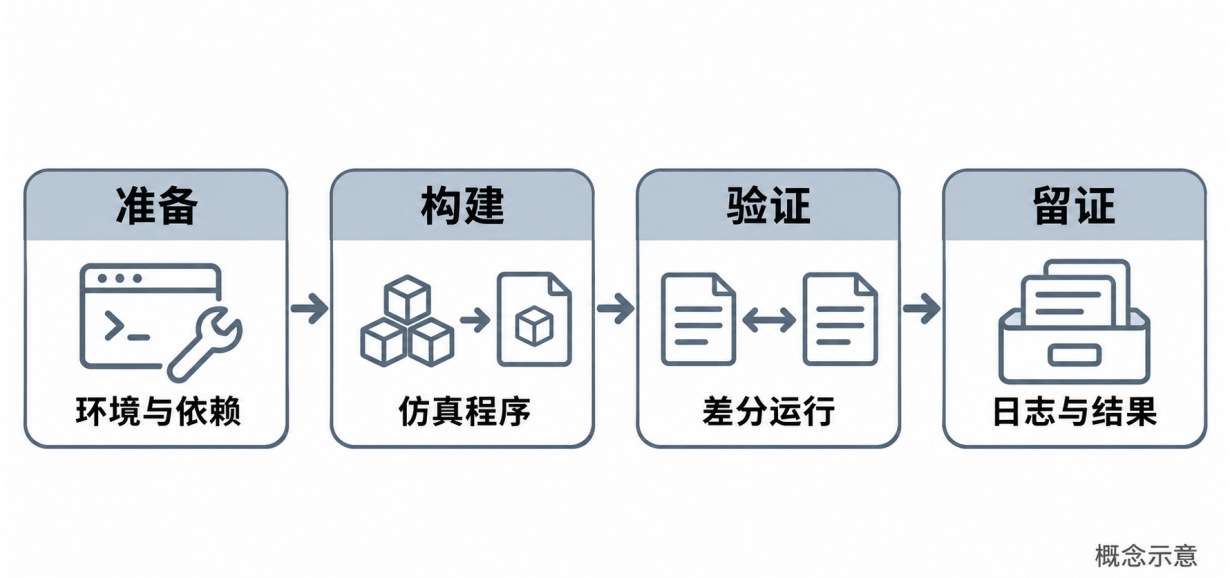
```
./configure CC=/usr/local/llvm/bin/clang CXX=/usr/local/llvm/bin/clang++
LINK=/usr/local/llvm/bin/clang++
make -j2
sudo make install
```

安装后回到团队仓库根目录，运行 activate.sh 和 check-environment.sh。原环境目录为/home/riscv/riscv-lab/xs-env/XiangShan；activate.sh 将实际团队仓库设为 NOOP_HOME，依据 TEAM_XS_ENV_ROOT 选择 NEMU 和 AM，并设置 LINUX_GNU_TOOLCHAIN=1。向量工作负载构建显式使用`MARCH=rv64gcv`。

1.2.3 集成构建与操作记录

确认源码、工具和输出目录后，依次构建参考库、EMU、目标程序及共同 flash，再执行差分运行：

```
source team/scripts/activate.sh
bash team/scripts/check-environment.sh
bash team/scripts/stage1.sh nemu
bash team/scripts/stage1.sh emu
TEAM_ID= 正式团队标识 bash team/scripts/stage1.sh workload
bash team/scripts/stage1.sh run
```



概念示意：部署、运行与证据归档。技术依据见图 1-2。

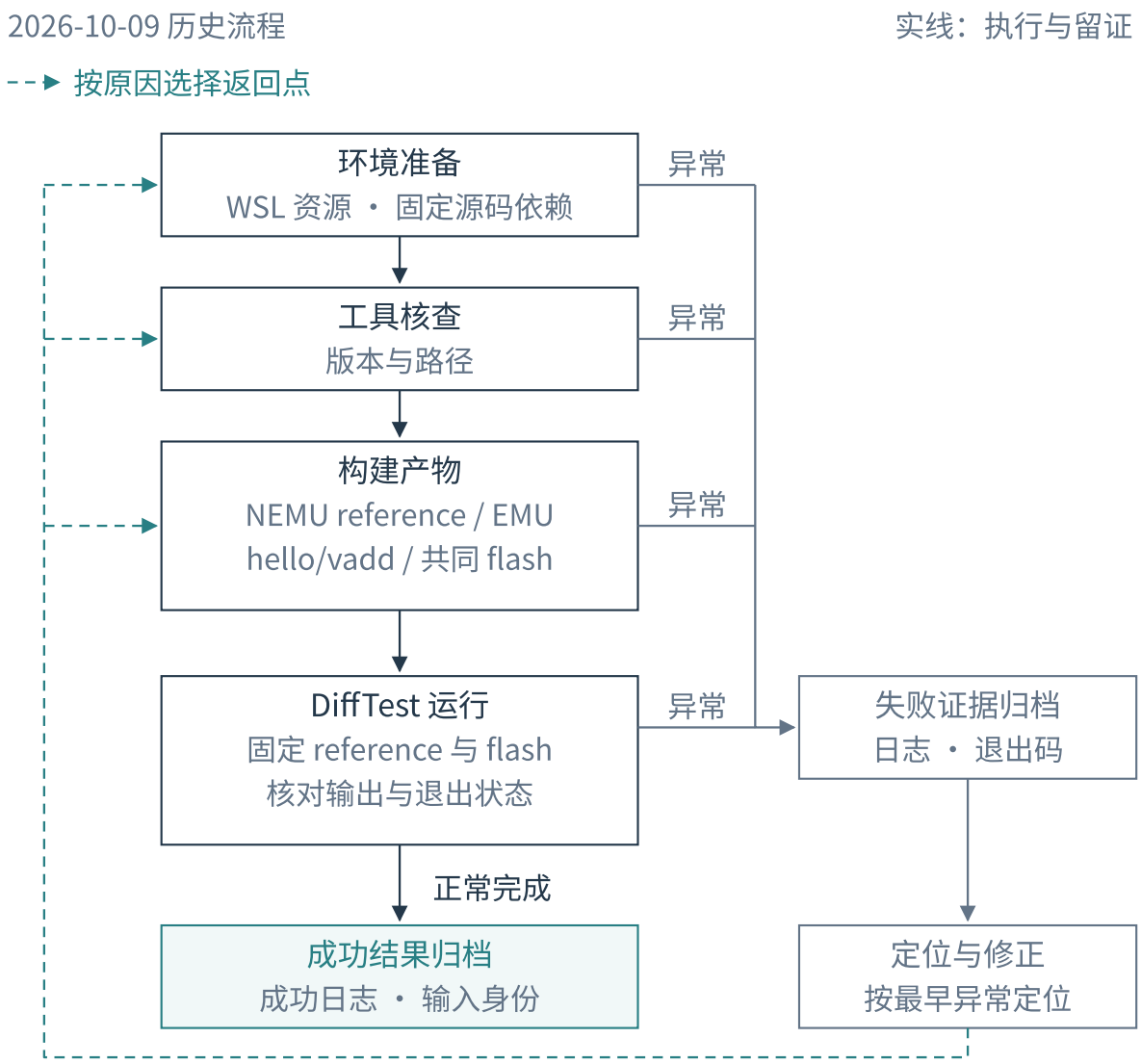


图 1-2　环境部署与验证执行流程

环境准备、工具核查、构建、DiffTest 运行和证据归档构成主流程。失败记录单独保存，定位后按原因返回相应环节。

10 月 9 日构建按实际资源调整并行度，过程保留在原始日志中。下表时间沿用日志的 UTC+8 时区。

表 1-2　2026 年 10 月 9 日主要操作记录

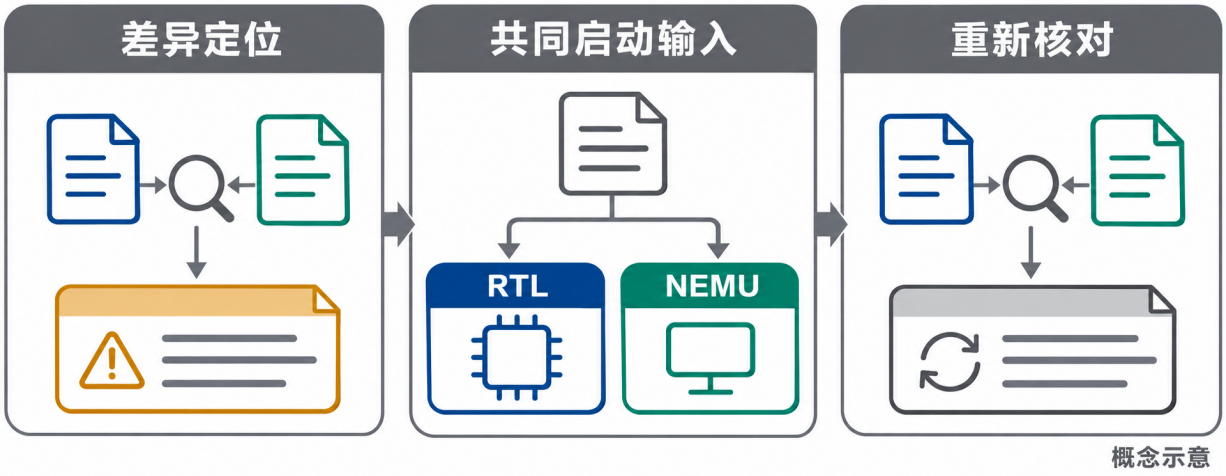
时间	操作与结果
20:42:15	中断并恢复 C++ 构建，并行度调整为 4，保留已完成对象。
20:53:32	EMU 构建完成，状态文件记录 exit_code=0。
20:55:26	工作负载与共同 flash 构建完成，记录 WORKLOAD_BUILD_PASS。
21:00:28	正常差分运行结束，退出状态为 0。

编译并发用于分配宿主资源，DUT 核数和 EMU 仿真线程数由模型配置决定。原 C++ 构建的`-j4` 与工具安装示例的`-j2` 对应不同步骤。复用对象文件时，源码、工具与生成参数随构建记录一起保存；长作业另记录资源峰值、日志大小、墙钟限额和进程结束状态。

1.3 共同启动输入与差分故障定位

初次运行在 PC0x10000004 出现 t0 寄存器差异，记录的 instrCnt 为 2，尚处于启动代码。比较默认 flash 和反汇编后发现，两端分别使用了 3 条与 6 条启动指令。修复由同一份 boot-flash.S 生成 12 字节 boot-flash.bin，并通过运行参数显式提供给 DUT 与 NEMU。随后两端以共同启动输入进入同一工作负载，在 DiffTest 启用的条件下完成 hello/vadd 检查。

ELF 保存符号、装载段和调试资料，BIN 提供程序字节，flash 提供复位后的启动代码。定位波形时，用与被测 BIN 同次构建的反汇编核对 PC 和机器字；链接参数或库布局改变后，应重新建立这组对应关系。flash 中的文件偏移与处理器执行地址通过各端装载映射关联。



概念示意：统一启动输入与差分恢复。技术依据见图 1-3。

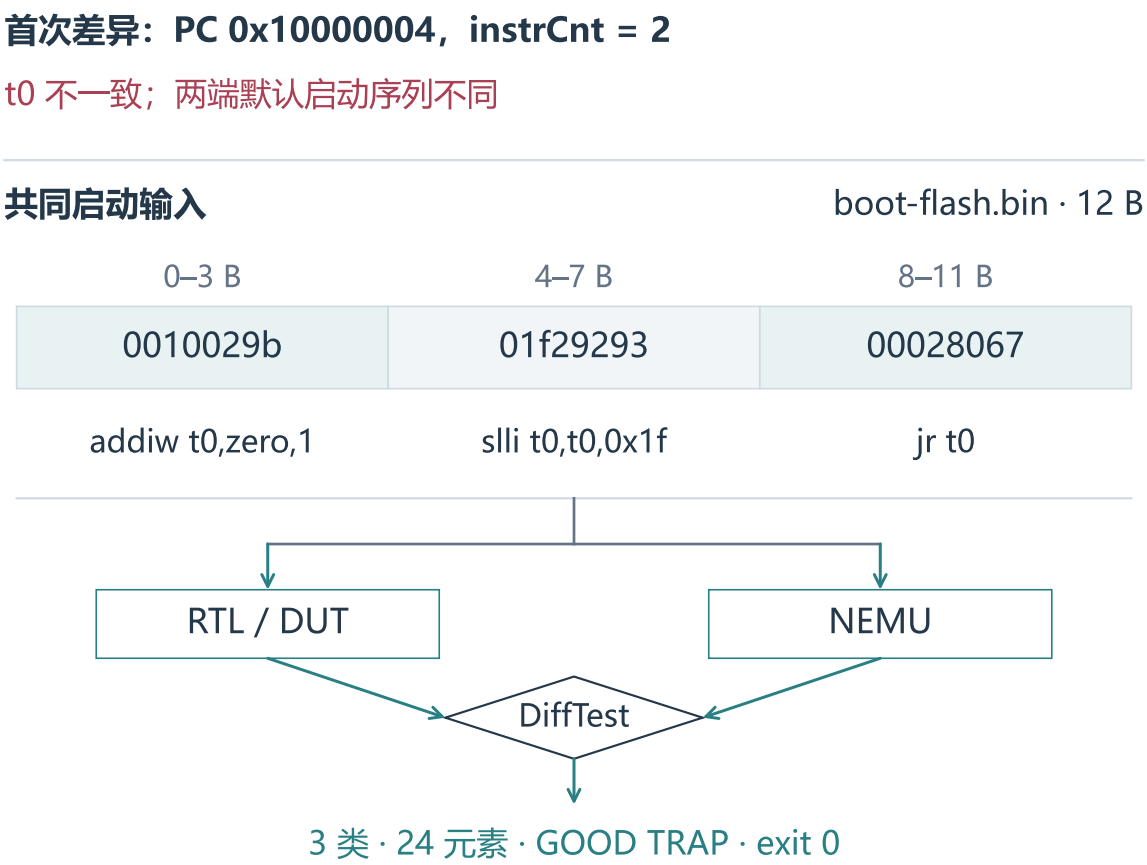


图 1-3　共同启动镜像与差分恢复

首次差异位于 PC 0x10000004、instrCnt=2。共同启动文件由三条机器指令组成，图中 0–3、4–7 和 8–11 标示文件字节偏移。显式加载同一 12 字节 flash 后，三类共 24 个元素完成比较，并以 GOOD TRAP、exit 0 结束。

1.4　hello 输出与三类 vadd 结果

1.4.1　输入配置与结果解释

vadd 程序设置 SEW8、LMUL1、VL8，执行两次`vle8.v`、一次`vadd.vv`和一次向量存储，再逐字节比较标量参考。三个用例检查基本加法、负数补码和 8 位回绕，合计比较 24 个活动元素。

表 1-3　vadd 输出的两种表示

用例	按元素顺序输出的字节	结果低 64 位
正数相加	9 9 9 9 9 9 9 9	0x0909090909090909
负数补码	254 252 250 248 246 244 242 240	0xf0f2f4f6f8fafcfe
边界回绕	128 127 0 0 0 0 254 254	0xfefe000000007f80

每个元素按模 256 保留低 8 位。例如打印值 254 对应十六进制 FE，按 8 位补码解释为 −2。波形将多个字节合并显示为十六进制，右端为最低字节；日志按元素序号递增输出，因此两种书写顺序应按字节位置对应。VLEN128 给出寄存器宽度，VL8 给出本用例的活动元素数，表中结果覆盖低 64 位的八个元素。

1.4.2 10 月 9 日环境验证

10 月 9 日 hello 输出使用 riscv-vdot-team，身份状态为 placeholder，显示的 IPv4 为 10.198.93.97，对应当时 WSL 宿主。正常记录包含 HELLO_VADD_PASScases=3lanes=24、DiffTest 启用标记、GOOD TRAP 和退出状态 0；整程序统计为 16557 条指令、42962 周期，覆盖启动、UART 和结果校验。

Historical hello/vadd output; original run 2026-10-09

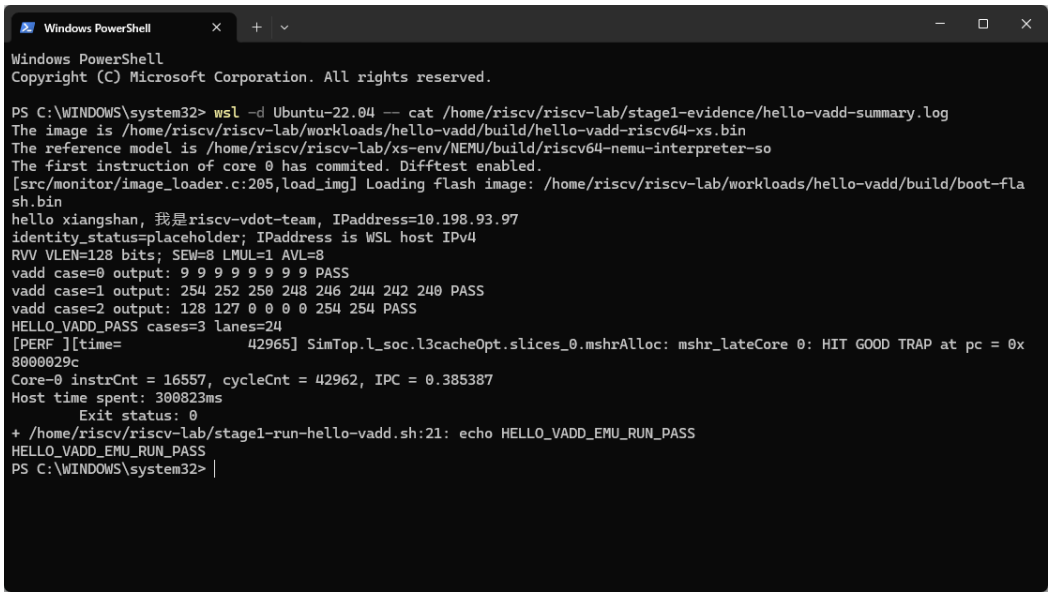


图 1-4 hello 与 vadd 正常结束输出

日志展示三类 vadd 结果、DiffTest 运行和 GOOD TRAP 终态，输出中的身份与地址按原实验保留。

1.4.3 10 月 25 日双模型复核

10 月 25 日（UTC+8）的 T04 实验以个人标识 LiZufan001 构建英文 hello 程序，输出明确标注地址来自构建宿主：

```
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
```

同一 BIN 分别在封存生产 EMU 和独立 FST 模型上运行，均加载普通 NEMU 参考库并启用 DiffTest。两次运行都通过三组、24 个元素检查，命中 GOOD TRAP 并正常退出；每次为 16621 条指令、43249 周期。这里的周期同样对应完整 hello/vadd 程序。表内实际起止时间来自各自运行回执。

表 1-4 T04 双模型运行记录（2026-10-25，UTC+8）

模型	实际起止时间	运行结果
生产 EMU	01:09:21.574716–01:10:18.357888	DiffTest 通过，GOOD TRAP，exit 0
独立 FST 模型	01:28:57.974329–01:31:24.009122	DiffTest 通过，GOOD TRAP，exit 0

新 FST 模型复用 T17 的 1807 个逐文件核验输入；最终核对原件及副本均无变化。发布源码重新编译得到的 BIN 和 flash 与实际执行输入逐字节相同，原始 ELF 和发布 ELF 分别留存。对应验收记录、生产运行回执和FST 运行回执均随证据保存。

1.5 向量执行与波形观察

向量指令经取指、译码、重命名与调度进入执行单元。两个`vle8.v`提供输入，`vadd.vv`产生结果，随后经过写回与 ROB 提交。本文用提交 PC0x80000138、机器字 0x021101d7 及`vecwen=1`，`wdest=3`关联三类用例；执行结果出现和架构提交分别作为观察点记录。

1.5.1 10 月 9 日事件与原始截图

以下数值沿用原 FST 提取 CSV 的事件坐标。局部截图未显示时钟，显示单位为 ps；本表用于定位输入、输出和提交，指令时延与性能通过专门测量窗口评估。

表 1-5 10 月 9 日 vadd 事件坐标

用例	输入	输出	提交	输出低 64 位
正数	19625	19626	19631	0x0909090909090909
负数	25460	25461	25466	0xf0f2f4f6f8fafcfe
边界	33009	33010	33015	0xfefe000000007f80



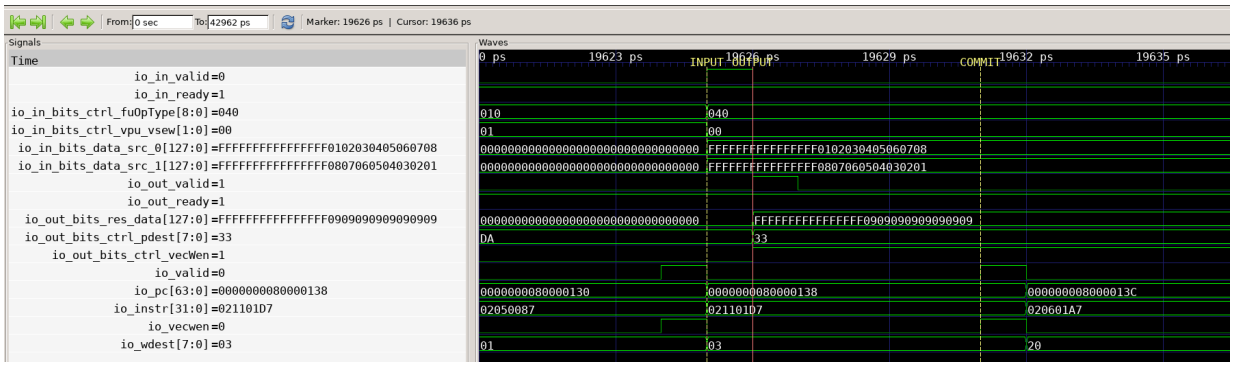
概念示意：向量操作的执行与状态观察。技术依据见图 1-5。



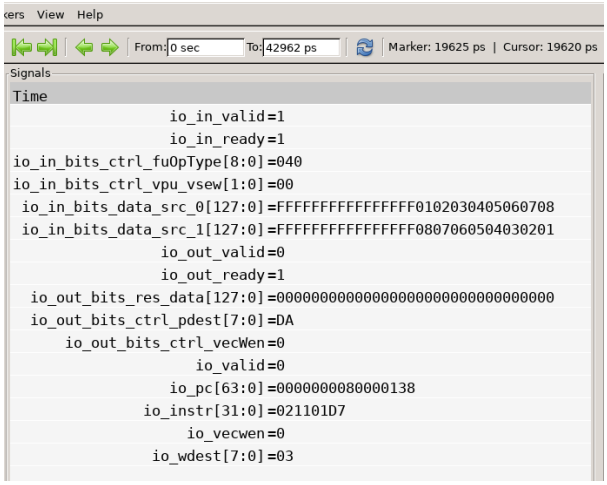
图 1-5 vadd 输入、输出与提交事件时间线

三类用例按指令身份关联执行端输入、结果输出和 ROB 提交；数值沿用事件 CSV 坐标。

(a) Output handshake, marker 19626 ps; all three event positions visible



(b) Input marker 19625 ps



(c) Commit marker 19631 ps

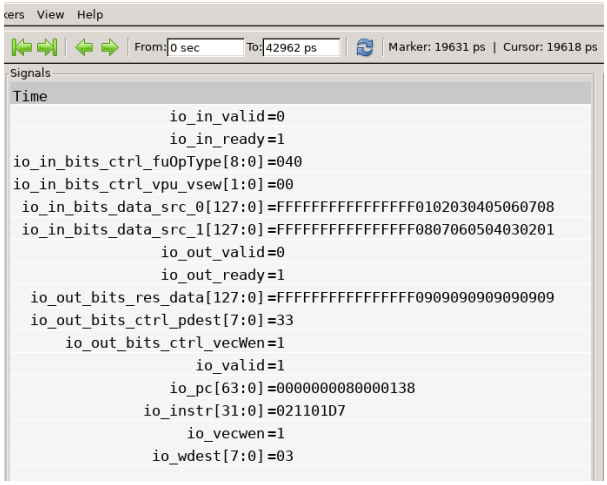
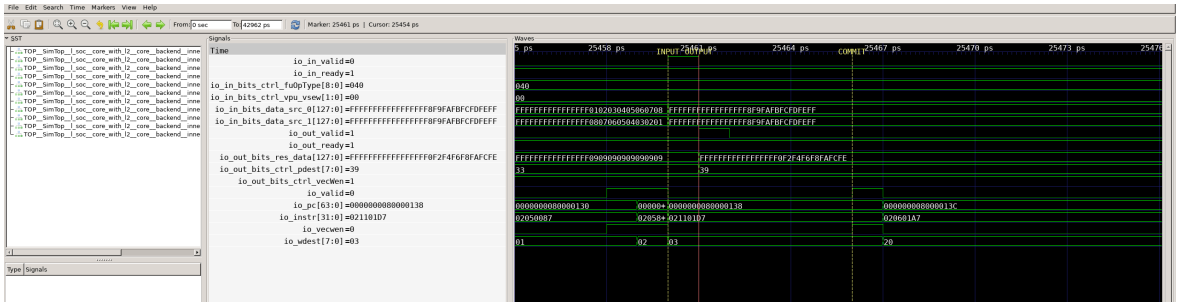


图 1-6 vadd 正数用例的执行与提交波形

(a) 输出事件及相关位置；(b) 输入游标处信号值；(c) 提交游标处信号值。19625、19626 和 19631 对应表中的三个事件坐标。

(a) Negative case: output marker 25461 ps



(b) Boundary case: output marker 33010 ps

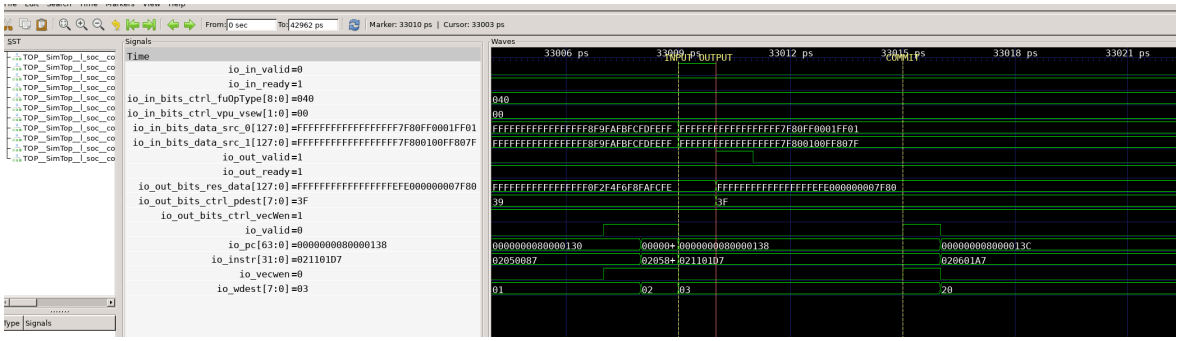


图 1-7 vadd 负数与边界用例结果波形

负数和边界样例输出低 64 位分别为 F0F2F4F6F8FAFCFE 与 FEFE000000007F80，游标对应各自输入、输出和提交位置。

原完整 FST 路径记录为/home/riscv/riscv-lab/stage1-evidence/hello-vadd.fst，大小 220295598 字节。复核可先从提交 PC 定位同次反汇编，再根据 CSV 打开波形窗口，最后联合查看时钟、valid/ready、操作数、结果及事务标识。信号层级与位宽由 evidence/wave-signals.json 提供。

1.5.2 10 月 25 日含时钟的调试记录

T04 的独立 FST 模型保留实际时钟信号，最终选择两个 VialuFix、八个提交槽和 127 个信号，检查 33249 个完整上升沿，得到三次 FU 输入、三次 FU 输出和三次 DPIC 提交观测，共九条事件。三个运算均在 exus_2.VialuFix 发生，同时检查了 exus.VialuFix。

采样取每个局部上升沿前的稳定低相值，周期字段取当时的 DifftestTrapEvent.io_cycleCnt，FST 原始时戳另存。DPIC 记录经过 Rob.scala 中`delay=3` 的观察路径，表内 DPIC 列因此表示提交接口的观测时刻。

表 1-6 T04 按时钟沿采样的事件记录

用例	FU 输入 timer	FU 输出 timer	DPIC 观测 timer	ROB flag/value/pdest
正数	19525	19526	19531	1 / 13 / 51
负数	25526	25527	25532	0 / 21 / 57
边界	33193	33194	33199	1 / 25 / 63

三类数值结果与表1-3一致。完整采样定义、信号清单、原始时戳及摘要保存在wave.json；选通信号 FST 可直接用 GTKWave 查看。接口与观察路径分别对应固定源码的VIAluFix 包装

层和Rob.scala。

工具检查共 24 项，覆盖日志完整性、实际退出状态、输入身份、关键结果，以及缺 FU、缺半周期、未知有效操作数和错误事务标识等反例。首次 FST 构建的工具路径问题和首版提取器遗漏无数字后缀 FU 的记录均保留；修正提取范围后，从同一 FST 完成最终九事件核对。这些检查验证记录与提取工具，实际硬件输入仍为上述三组 vadd 用例。

1.6 阶段结论与复现索引

阶段一完成了环境部署、共同启动输入配置、三类向量加法的差分验证和可定位的波形观察链。10 月 9 日记录说明环境建立过程，10 月 25 日记录补充配置身份的英文 hello 输出与含时钟的执行观察。后续阶段在这一环境上开展 vdot 语义、RTL 和应用性能验证。

1.6.1 源码与工具版本

下表为 10 月 9 日部署时固定的源码身份，完整递归依赖见 evidence/submodules-after-build.lock.txt。

表 1-7 10 月 9 日源码版本锁

组件	提交号
xs-env	b0b506481e839a5df8a580e76f5aa246dc60fe7c
香山基线	16ae9ddcda54fc9a2fddffad73174cf793ac7814
NEMU	a48a11f208ff81ad765591d83271d1b339fb3861
nexus-am	086a238062fdfb3281b745a537b361983d0154b5
DRAMsim3	1d2a9bf6da8e050d975ef2669c42faa93943f489
DiffTest	62e4540c03c03aa947bdb67f4b6bf66f1a29c5d0
yunsuan	cadd3c2f43096e253d61296b33ac697be8354e29

表 1-8 10 月 9 日工具版本

工具	版本
Oracle GraalVM JDK / javac	21.0.12
LLVM / Clang	19.1.7
Mill	项目版本 0.12.3
Scala / Chisel	2.13.15 / 6.6.0
firtool	1.62.1
Verilator	5.052，提交 ea338be98e1e838d3518809ce8899f85a009963c
RISC-V GCC / GNU binutils	11.4.0（riscv64-linux-gnu-前缀）/ 2.38
宿主 GCC / G++	11.4.0
Make / CMake / Ninja	4.3 / 3.22.1 / 1.10.1
Python / GTKWave	3.10.12 / 3.3.104

完整版本输出见 evidence/tool-versions.txt；Ubuntu 软件包清单保留原环境的软件来源。新环境按所需工具安装并核对版本，硬件改动的验证结果再绑定各自构建产物。

1.6.2 T04 产物身份

10 月 25 日 T04 运行使用的 T17 硬件生成源提交为 34bae6d4acceac939f958199dae53c14bc1f6183。以下摘要直接对应 evidence/stage1-current-20261025/中的运行和验收记录。

表 1-9 T04 主要产物 SHA-256

对象	SHA-256
生产 EMU	ff840528eb1009e94906eefb3773ca321050f309553543ab26f0db23c410e30f
FST 模型	1ff3f457ff582d0c3155a15dafa1fa60b8275d9356e29a0b7806c9d3915ce9f7
普通 NEMU 参考库	8af118e5519db7ef6001010a2d547707c5d47630412e80abfc3f343bfef987c9
共同程序 BIN	d81437b44473cef2b0c5777330e13fd076567d4fc1a4296be35d971e51a4dcad
共同 flash	a3360212cf7bb784119c1719c9258362f8567803601b2359aee2c104864fa994
完整 FST（181143183 字节）	cf8abe9104fb955c7dc4bf1efda6588a61060fce33857b9ddc6d5020421be299
选通信号 FST（393288 字节）	4c9f442704871010026654fd9d60854024b270a83be6354be5dee3ff25df24e1

1.6.3 文件与记录索引

运行源码位于 team/workloads/hello-vadd。10 月 9 日证据归档位于 team/evidence/stage1-20261009，T04 证据位于 team/evidence/stage1-current-20261025；报告源码的 evidence/目录保存用于本文的对应日志与索引。完整 FST 按记录中的路径和摘要定位。

表 1-10 报告附件索引

报告目录中的记录	用途
evidence/workload-build.log	工作负载、flash 构建命令与成功状态。
evidence/hello-vadd-summary.log	10 月 9 日身份、三组结果、DiffTest 和退出摘要。
evidence/rendered/nemu-build.txt	NEMU 构建日志的可读副本。
evidence/emu-build-20261009-204228.log	EMU 构建与并行恢复过程。
evidence/tool-versions.txt	工具版本原始输出。
evidence/submodules-after-build.lock.txt	递归依赖完整提交锁。
evidence/vadd-commit-events.csv	配置、加载、vadd 及后续指令的提交记录。
evidence/vector-fix-alu-events.csv	三组向量 ALU 输入与输出记录。
evidence/wave-signals.json	10 月 9 日信号层级、位宽与标识。
evidence/stage1-current-20261025/production-run/run.json 与 trace-run/run.json	T04 两次运行的命令、输入前后摘要、退出状态和日志摘要。
evidence/stage1-current-20261025/wave/wave.json	T04 采样定义、127 个信号、九条事件及波形摘要。

报告目录中的记录	用途
evidence/stage1-current-20261025/acceptance.json 与 RESULT.md	T04 输入等价、24 项工具检查与完整复现索引。

复核时先检查运行回执和通过摘要，再核对程序、reference 与 flash 身份，最后用反汇编和事件 CSV 定位波形。详细操作入口见阶段一记录、环境部署方法和T04 复现说明。

1.7 操作日志摘录

本节保留主要运行摘要与构建完成片段；完整构建日志、版本锁和事件 CSV 随源码包提供，路径见表1-10。

1.7.1 10 月 9 日工作负载构建

```
+ /home/riscv/riscv-lab/stage1-build-workload.sh:20: make ARCH=riscv64-xs
MARCH=rv64gcv -j1
# Building hello-vadd [riscv64-xs] with AM_HOME
{/home/riscv/riscv-lab/xs-env/nexus-am}
+ CC main.c
# Building lib-am [riscv64-xs]
+ AS src/nemu/common/mainargs.S
+ AR -> build/am-riscv64-xs.a
# Building lib-klib [riscv64-xs]
# Creating binary image [riscv64-xs]
+ LD -> build/hello-vadd-riscv64-xs.elf
+ OBJCOPY -> build/hello-vadd-riscv64-xs.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:21: riscv64-linux-gnu-gcc -c
-march=rv64gc -mabi=lp64d boot-flash.S -o build/boot-flash.o
+ /home/riscv/riscv-lab/stage1-build-workload.sh:22: riscv64-linux-gnu-objcopy
-O binary -j .text build/boot-flash.o build/boot-flash.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:23: riscv64-linux-gnu-objdump
-d build/boot-flash.o


+ /home/riscv/riscv-lab/stage1-build-workload.sh:28: echo WORKLOAD_BUILD_PASS
WORKLOAD_BUILD_PASS
+ /home/riscv/riscv-lab/stage1-build-workload.sh:1: rc=0
++ /home/riscv/riscv-lab/stage1-build-workload.sh:1: date -Is
+ /home/riscv/riscv-lab/stage1-build-workload.sh:1: printf 'exit_code=%s
end=%s\n' 0 2026-10-09T20:55:26+08:00
```

1.7.2 10 月 9 日正常运行摘要

```
The image is
/home/riscv/riscv-lab/workloads/hello-vadd/build/hello-vadd-riscv64-xs.bin
The reference model is
/home/riscv/riscv-lab/xs-env/NEMU/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
[src/monitor/image_loader.c:205,load_img] Loading flash image:
/home/riscv/riscv-lab/workloads/hello-vadd/build/boot-flash.bin
hello xiangshan, 我是 riscv-vdot-team, IPaddress=10.198.93.97
identity_status=placeholder; IPaddress is WSL host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
```

```
HELLO_VADD_PASS cases=3 lanes=24
[PERF ][time=               42965] SimTop.l_soc.l3cacheOpt.slices_0.mshrAlloc:
mshr_lateCore 0: HIT GOOD TRAP at pc = 0x8000029c
Core-0 instrCnt = 16557, cycleCnt = 42962, IPC = 0.385387
Host time spent: 300823ms
        Exit status: 0
+ /home/riscv/riscv-lab/stage1-run-hello-vadd.sh:21: echo
HELLO_VADD_EMU_RUN_PASS
HELLO_VADD_EMU_RUN_PASS
```

1.7.3 10 月 25 日生产模型输出

```
emu compiled at Oct 24 2026, 09:22:45
Using simulated 32768B flash
use /home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/」
boot-flash.bin as flash bin
Using simulated 128MB RAM
The image is /home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/worklo」
ad/build/hello-vadd-riscv64-xs.bin
The reference model is /home/lizufan/riscv-lab/t17-clean-20261024-0854/support」
-01/nemu/reference/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
[src/monitor/image_loader.c:205,load_img] Loading flash image: /home/lizufan/r」
iscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin

[src/monitor/image_loader.c:250,load_img] Read 12 bytes from file /home/lizufa」
n/riscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin to
0x0xf0000000
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
Core 0: HIT GOOD TRAP at pc = 0x800002a4
Core-0 instrCnt = 16621, cycleCnt = 43249, IPC = 0.384309
Seed=0 Guest cycle spent: 43253 (this will be different from cycleCnt if emu
loads a snapshot)
Host time spent: 56742ms
```

1.7.4 10 月 25 日 FST 模型输出

```
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
Core 0: HIT GOOD TRAP at pc = 0x800002a4
Core-0 instrCnt = 16621, cycleCnt = 43249, IPC = 0.384309
Seed=0 Guest cycle spent: 43253 (this will be different from cycleCnt if emu
loads a snapshot)
Host time spent: 145975ms
```